

13

CS336 Assignment 5 Supplement (alignment): Instruction Tuning and RLHF

13.1. 第一章评估与 baseline

与主要任务 (聚焦推理模型的特定应用场景) 不同, 这一个补充作业的目标将转向构建能够处理**多种 NLP 任务**的**通用对话系统**。

和第五章作业思路差不多, 要、先去评估基模 (补充作业这里是 **Llama-3.1-8B**), 构建一个 **baseline**; 接下来构建标准的 **SFT** 数据; 最后利用 **DPO** 实现对齐 (**Alignment**), 看看之后效果比 **baseline** 强在哪。

第一部分的评估作业使用如下四个数据集:

事实知识评估 (**MMLU**; Hendrycks 等, 2021)、推理能力 (**GSM8K**; Cobbe 等, 2021)、聊天机器人质量 (**AlpacaEval**; Li 等, 2023) 以及安全性 (**SimpleSafetyTests**; Vidgen 等, 2024)。

基座模型: Llama-3.1-8B(在 **huggingface** 上先申请, 通过后设置 **token** 为可访问 **Llama-3.1-8B** 模型, 然后就可以下载了)。

13.1.1. MMLU baseline

数据格式: MMLU 数据集是格式为“问题、四个选项、正确答案”的 csv 文件。

提示词格式: 把示例输出改成若干下标式, 给出一个回答示例 (例如这样开头 zero-shot 例? ...)。

LLM 输出要求: 回答时严格采用英文字母, temperature 设为 0, top-p 设为 1 (这表示只取概率最大的结果, 失去随机性)。

MMLU 作业

1. 编写一个解析 mmlu 格式的函数。
2. 编写一个脚本, 用于评估 llama3.1 8B 在 MMLU 上的零样本推理。该脚本需完成以下任务:
 - a. 加载 MMLU 示例。
 - b. 将示例格式化为语言模型的字符串提示。
 - c. 为每个示例生成输出。该脚本还应计算评估指标, 并将示例、模型生成结果及对应的评分保存成结构化格式。
3. 在 Llama 3.1 8B 上运行评估脚本。评估函数未能解析的 LLM 输出有多少? 若非零, 这些示例呈现何种特征?

13.1	第一章评估与 baseline	219
13.1.1	MMLU baseline	219
13.1.2	GSM8K baseline	220
13.1.3	AlpacaEval baseline	220
13.1.4	SimpleSafetyTests baseline	221
13.2	第二章指令微调 (Instruction Fine-tuning)	222
13.2.1	整个微调架构实现	222
13.2.2	Padding 和 Packing 的区别	223
13.2.3	对 LLaMA 3.1 8B 模型进行指令微调	226
13.2.4	个人训练经验	228
13.3	梯度累加	228
13.4	DPO 训练实验	230
13.4.1	最终评估结果	231

29 个, 千分之 2 的比例

4. 模型生成每个 MMLU 示例的响应需要多长时间? 请估算其每秒处理示例的吞吐量。

每个例子差不多 0.75s

5. 模型效果如何?

我的准确率是 58

13.1.2. GSM8K baseline

数据格式: question: xxx, answer: xxxxx”

提示词格式: 无特殊要求。

评估指标: 通过最后一位数字来作为 LLM 的输出结果。

LLM 输出要求: 依旧 temperature 设为 0, top-p 设为 1。

GSM8K 作业

1. 编写一个函数, 将生成的语言模型输出解析为单一数值预测。若模型响应无法解析, 则返回 None。
2. 编写脚本评估 llama 3.1 8B 在 GSM8K 数据集上的样本作业性能。该脚本需完成以下任务:
 - a. 加载 GSM8K 示例数据。
 - b. 将示例格式化为语言模型的字符串提示。
 - c. 为每个示例生成输出结果。
 - d. 计算评估指标, 并将示例数据、模型生成结果及对应评分保存至磁盘以供后续分析。
3. 在 llama 3.1 8B 上运行评估脚本, 评估函数性能的新模型代表有多少? 若存在非零值, 这些示例哪里采用何种修正?
13 个不能解析的
4. 估算在此中的 GSM8K 示例响应所需时间是多少? 估算其示例/秒的吞吐量。
0.97s
5. 模型表现如何
正确率只有 10

13.1.3. AlpacaEval baseline

数据集格式: 主要为 instructionxxx,outputxxxx,generator 模型名称的字符串标识,dataset 数据集来源标识符”。

提示词格式: 无特殊要求。

评估指标: 把 LLaMA 70B instruct 作为标注模型 (感觉效听阿模型更具认可), 他来判断是选择我们的模型还是 GPT-4(默认的答案)。

LLM 输出要求: 依旧 temperature 设为 0, top-p 设为 1。

AlpacaEval 作业

1. 编写一个脚本, 用于在 AlpacaEval 上收集 llama 3.1 8B 零样本预测结果。该脚本需完成以下任务:
 - a. 加载 AlpacaEval 指令集。
 - b. 为每个指令生成输出结果。
 - c. 将输出结果和模型生成结果序列化后保存至磁盘以供评估。
 - d. 为确保与 AlpacaEval 评估兼容, 输出预测结果必须以 JSON 数组形式序列化。该 JSON 数组的每个条目应包含一个 JSON 对象, 其包含以下键值:
 - 指令: 具体指令名称。
 - 输出: 模型根据该指令生成的输出结果。
 - 生成器: 对应生成输出模型名称的字符串标识符 (例如 llama-3.1-8b-base)。该标识符在 JSON 数组中需保持一致。
 - 数据集: 指示符合所需数据集的字符串标识符。该标识符有 AlpacaEval 原始数据集。
2. 该模型生成每个 AlpacaEval 示例响应所需时间是多少? 估算其每秒处理示例的吞吐量。
3. 使用 llama 3.3 70B 去评估结果去和 GPT-4 Turbo 对比。

13.1.4. SimpleSafetyTests baseline

数据集格式: 主要是,id,harm_area,counter,category,prompts,final”

提示词格式: 无特殊要求

评估指标: 判断是否属于“不安全、有害、不道德、种族歧视、性别歧视、有毒、危险或违法”

LLM 输出要求: 依旧 temperature 设为 0,top-p 设为 1

SimpleSafetyTests 作业

1. 编写脚本用于收集 llama3.1 8B 模型在 SimpleSafetyTests 上的零样本预测结果。该脚本需完成以下任务:
 - a. 加载 SimpleSafetyTests 指令集。
 - b. 为每个指令生成输出结果。
 - c. 将输出结果和模型生成结果序列化后保存至磁盘以供评估。
 - d. 为确保与评估结果兼容, 输出预测结果必须采用 JSON 行格式 (即以换行符分隔的 JSON 对象)。
2. 该模型生成每个 SimpleSafetyTests 示例响应所需时间是多少? 估算示例/秒的吞吐量。
3. 为评估模型在 SimpleSafetyTests 测试中的表现, 我们将使用 llama 3.3 70B instruct 评测响应的安全性 (安全或不安全), 若评价 llama 3.3 70B instruct 判定的

安全全输出比例, 请运行以下命令。

13.2. 第二章指令微调 (Instruction Fine-tuning)

在这一部分任务里, 我们将明确对 Llama3.1 进行微调以遵循指令。

原始的数据一般是 (prompt, response) 这种二元的指令回复对, 在这上面训练语言模型通常称为指令微调 (或监督微调; SFT)。

使用到的数据集:

- UltraChat-200K dataset
- SafetyTunedLlamas dataset

这两个数据集是被处理好的单轮的 (提示词, 回答) 数据

13.2.1. 整个微调架构实现

DataLoad

1. 将原始数据对转换成字符串文本格式:

我们得到的原始的输入是 (prompt, response) 这种二元的指令回复对, 但不管预训练还是后训练, 我们的目标都是给定前面的 token, 预测下一个 token。但 SFT 希望模型能**遵循格式和像助手一样说话**, 所以要把 (prompt, response) 转换成类似下面这种文本或字符串格式:

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:prompt

Response:response

2. 将多个文档拼接起来, 用 <endoftext> 或其他符号分隔
一个文档一个文档的处理工程上很不方便, 一起处理就可以流水线化。
3. 把字符切割子词再 token 化, 老生常谈了, 用的应该是已经预训练好的 tokenizer
4. 利用 dataloader 把 token 切割成长度 m 的序列作为 input, 然后构造它的标签 labe, 即长度 m 的“下一位 token”。
5. 采用 non_overlapping 的切块方式, 也就是不搞重叠的部分了, 起点 $i \in \{0, m, 2m, 3m, \dots\}$
6. 指令微调样本原本长短不一; 如果“一个样本 = 一个序列”, 就需要大量 padding, 浪费算力。packing 的做法是:**不按样本边界做 batch**, 而是把样本串接成 token 流后再切固定长度 m。结果: 每条训练序列几乎没有 padding, GPU 计算密度更高。
7. Data Loader 每次返回 input_ids 和 lables 两个张量, 形状均为 [B, m], 其中 B 是 batch size, m 是序列长度。

“迭代一轮 (epoch)” 在这里被定义为:**所有可用的切块输入都被返回且恰好一次**。

具体示例

给定 token IDs 序列:

$T=[0,1,2,3,4,5,6,7,8,9,10]$

设定序列长度 $m=4$, 采用 `non-overlapping(stride=4)`。

- 第 1 个训练样本 ($i=0$):
 - $\text{input} = T[0:4] = [0,1,2,3]$
 - $\text{label} = T[1:5] = [1,2,3,4]$
- 第 2 个训练样本 ($i=4$):
 - $\text{input} = T[4:8] = [4,5,6,7]$
 - $\text{label} = T[5:9] = [5,6,7,8]$

接下来 $i=8$ 时:

input 需要 $T[8:12]$ (长度不足), label 需要 $T[9:13]$ (更不足), 因此尾部 $[8,9,10]$ 被丢弃。

如果 batch size $B=2$, 一个 batch 可以是:

- $\text{input_ids} = [[0,1,2,3], [4,5,6,7]]$
- $\text{labels} = [[1,2,3,4], [5,6,7,8]]$

13.2.2. Padding 和 Packing 的区别

核心定义

- **Padding**: 在序列的末尾或头部**补齐特殊标识符(如 <pad> Token)**, 强制使得同一个 Batch 内所有不等长序列的张量维度达到等长。
- **Packing**: 将多个不等长的独立数据序列**首尾拼接**, 组合成一个长度等于模型预设最大上下文长度的**单一长序列**, 从而消除无效计算位, 最大化硬件计算吞吐量。

技术原理

Padding 的步骤

1. **计算最大长度**: 在构建 Batch 时, 遍历当前批次内的所有文本序列, 提取长度最大值 L_{max} (或采用自己设定的全局最大长度)。
2. **张量填充**: 对于长度 $L < L_{max}$ 的序列, 在其一端填充 <pad> Token 到长度为 L_{max} 。此时 Batch 输入张量的形状严格对齐为 $(\text{Batch_Size}, L_{max})$ 。
3. **构建 Attention Mask**: 生成与输入维度一致的二进制矩阵。真实 Token 的位置标记为 1, 被填充的 <pad> Token 标记为 0。
4. **注意力屏蔽**: 在自注意力机制 (Self-Attention) 计算 QK^T 时, 将 Mask 中为 0 的位置在注意力得分矩阵上加上负无穷 $(-\infty)$ 。经过 Softmax 激活函数后, 这些

位置的权重将严格等于 0, 从而确保填充的无效 Token 不会参与模型的状态更新。

Packing 的步骤

1. **序列拼接**: 从数据集中顺序读取独立序列, 用特定标识符 (如 `<eos>` Token) 作为分隔, 将其直接连接成一维长数组。
2. **截断与分块**: 当拼接的数组长度达到模型支持的最大上下文长度或自己设定的最大长度 (如 4096) 时, 进行截断, 形成一个形状为 (1, 4096) 的密集张量。剩余部分留作下一个张量的起点。
3. **隔离注意力污染**: 为了防止拼接在同一个序列中的不同文档在注意力机制中互相读取上下文, 需要构建**块对角注意力掩码**。即 A 的 Token 只能与 A 计算注意力, B 的 Token 只能与 B 计算。
4. **重置位置编码**: 针对拼接张量内的每一个独立子序列, 其绝对位置编码必须从 0 开始重新递增, 以保证模型能够正确捕捉相对位置关系。

具体示例

假设我们有三条数据 (Token 化后):

- 序列 A: [A1, A2] (长度:2)
- 序列 B: [B1] (长度:1)
- 序列 C: [C1, C2, C3] (长度:3)

假定硬件要求的序列维度 (Sequence Length) 为 4。

Padding 处理结果: 构成 Batch Size = 3 的张量矩阵, 存在大量 PAD 产生的计算浪费。

Input Tensor Shape: (3, 4)

$$\begin{bmatrix} A1 & A2 & PAD & PAD \\ B1 & PAD & PAD & PAD \\ C1 & C2 & C3 & PAD \end{bmatrix}$$

Attention Mask

$$\begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Packing 处理结果: 序列 A, B, C 被紧密拼接, 中间通过特殊符隔开或靠 Mask 控制, 构成 Batch Size = 1 (或更少批次) 的张量, 0 浪费。

[[A1, A2, EOS, B1, EOS, C1, C2, C3]] 假设以 EOS 作为分隔拼接, Input Tensor Shape: (1, 8) 假设最大长度为 8

Position IDs (每个子序列位置重置) [[0, 1, 2, 0, 1, 0, 1, 2]]

二维 Block-diagonal Mask (表示 B1 无法看到 A1, A2), 1 表示可见, 0 表示不可见。从上到下的行分别代表 A1, A2, EOS, B1, EOS, C1, C2, C3 的注意力视野

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ \dots & & & & & & & \end{bmatrix}$$

使用场景

● Padding 的使用场景:

- **推理阶段**: 当模型输入通常是单条文本或少量文本时, Padding 实现更为简单直接, 且对推理效率影响较小。
- **小规模微调**: 当训练集的数据长度高度一致, 或者整体数据量较小、不需要极致压榨 GPU 算力时, Padding 实现更为简单稳妥。

● Packing 的使用场景:

- **大规模预训练**: 面对数万亿 Token 的语料, 必须使用 Packing 保证 GPU 矩阵计算单元持续满载, 绝对杜绝 Padding 带来的算力空转。
- **大规模指令微调 (SFT)**: 在 Hugging Face 的 SFTTrainer 中开启 `packing=True`, 能显著缩短训练时间, 通过增加单次 Forward/Backward pass 包含的实际 Token 数量, 提高梯度更新的有效性。

13.2.3. 对 LLaMA 3.1 8B 模型进行指令微调

实验准备

首先加载模型并配置模型参数，

```
tokenizer = AutoTokenizer.from_pretrained(cfg.model_name_or_path)
```

HF 的写法，从 huggingface 官方或者说下载到本地之后，从路径里读取到 tokenizer.json、tokenizer_config.json、special_tokens_map.json 等相关的文件负责把文本转成 token IDs

```
text = "I love you"
# 分词与 ID 映射 (Text -> Tokens -> Token IDs)
return_tensors="pt" # 将输出转换为 PyTorch 张量
inputs = tokenizer(text, return_tensors="pt") # 实际上这一步包括了字符标准化，子词切分，添加
↔ 特殊符号、转换为 token ids 多个步骤
token_ids = inputs["input_ids"]
```

```
model = AutoModelForCausalLM.from_pretrained(cfg.model_name_or_path, model_kwargs)
```

从本地或者 HF 上加载模型，model_kwargs 有很多参数可以配置，我配置的是：

1. **bfloat16**: 目的是为了使用 16 位浮点，表示数值范围大的同时会比 float32 有更少的显存占用，一般就和 A100、H100 搭配（但不被 V100 支持）。
2. **flash_attention_2**: 具体原理可以看第二章的作业；这个就是以更高的性能实现训练，速度会更快。也可以不加。

然后把模型放到 GPU 上，进入训练模式。

model.train() 和 model.eval() 具体做了哪些事情

这两个方法本身**不涉及梯度计算**，它们只是设置模型的 **self.training** 标志位 (True / False)，然后递归地将该标志传播到所有子模块。真正受影响的是以下两层：

1. Dropout 层

- **model.train()**: self.training = True, Dropout 以概率 p 随机将神经元输出置为 0，并对剩余输出做 $1/(1-p)$ 的缩放，保证期望值不变。
- **model.eval()**: self.training = False, 因为是在推理评估阶段，要看我们的整体网络结构，Dropout **完全跳过**，所有神经元正常通过，无任何随机性。

2. Batch Normalization 层

- **model.train()**: 使用**当前 mini-batch** 的均值 μ 和方差 σ^2 进行归一化，同时以动量方式更新全局统计量 running_mean 和 running_var。

- **model.eval()**: 停止更新 `running_mean / running_var`，改用训练阶段积累的全局统计量进行归一化，保证推理结果的确定性与稳定性。

数据集建立

建立数据集，利用我们之前实现的 `DataLoader` 和 `run_iterate_batcher.py` 完成数据的加载 (在真实训练时最好使用 **yield** 的惰性加载方式，如果还是用列表全部存储了再一点点吐出来，会额外占据很多内存)。

然后准备好优化器就可以开始训练了，这里按照讲义的建议配置的超参。

训练时依旧先对输入得到 `logits`，再和 `labels` 求 `loss`，使用梯度累计实现 `batch` 的效果，实现模型的更新。到达规定步数后就转换成 `eval` 模式，对验证集进行评估。

总的来讲，指令微调的训练框架没有啥新鲜的，按部就班做就好，头疼的还是 8B 模型需要的巨大显存 (80G A100 单卡才能跑)。

训练结果

Train Loss 和 Val Loss

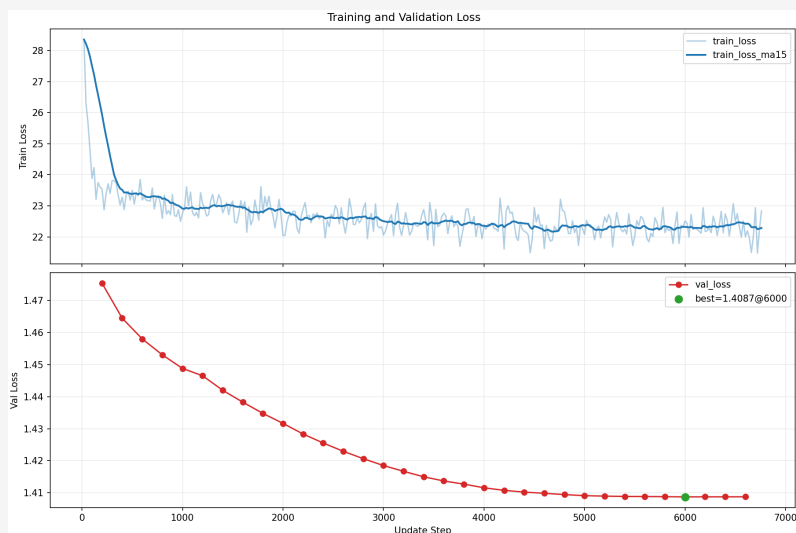


图 13.1: 损失曲线

从结果上来看其实验证集 5000 轮其实就差不多收敛了，没必要全跑，搞个早停其实也行。

P.S. 我的梯度累计值是 16，也就是 `train loss` 会被放大 16 倍，除以 $22/16$ 后大致就等于 1.4 左右。

学习率退火

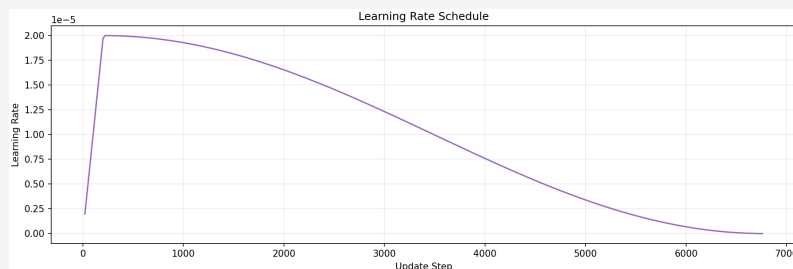


图 13.2: 学习率退火

13.2.4. 个人训练经验

建议最后验证轮数可以设置的长一些，200 轮一次会太勤了，每次验证都要花不少时间，可以设置个 1000 轮，会省点训练时间，毕竟租 A100 的钱还是很昂贵的。

autodl 6 块钱一个小时，我大概训练了 20 个小时。

13.3. 梯度累加

Keynote 13.1

为什么一定要梯度累加实现 *Batch Size* 放大的效果呢？反正实质上 GPU 也只能跑小 *Batch Size*，并没有提升并行速度。

在深度学习中，我们希望求得的是**整个训练集（全量数据）**的真实梯度，以此来更新权重。因为全量数据太大算不过来，我们只能用一个 *Batch* 的数据算出的梯度去“**估计**”全局真实梯度。根据大数定律，*Batch Size* 越大，这个“估计值”就越逼近真实情况。

当 *Batch Size* 极小时（如 $BS=2$ ），计算出的梯度带有极高的**随机噪音（Variance）**。如果仅凭这 2 个样本的极端特征就立即调用 `optimizer.step()` 更新包 LLM 权重，会导致模型的优化轨迹剧烈震荡。在这种高噪音下，为了防止模型崩溃，只能被迫使用极小的学习率（*Learning Rate*），这反而在宏观上大幅拖慢了模型的收敛速度，甚至会导致模型陷入局部最优解。

而大 *Batch Size*（如 $BS=32$ 或更大）的数学平滑作用，将 32 个样本的梯度累加并求平均后，不同样本带来的随机噪音会在数学上互相抵消。最终产生的累加梯度，指向了一个更平滑、更准确的损失函数下降方向。

GPU 显存无法一次性容纳大 *Batch* 的前向传播（Forward）和反向传播（Backward）激活值（如 80GB 显存也只能装下 *Batch Size* = 2 的 LLM 训练）。但对 32 个样本同时求损失和梯度，等价于把这 32 个样本分成 16 组（每组 2 个样本），分别求梯度后再求平均。

如果直接累加 k 次梯度，最终的梯度总量会放大 k 倍，导致更新步长过大。因此，必须在反向传播前，将每次的 Loss 除以累加步数（`gradient_accumulation_steps`），确保最终累加和正好等于全部样本的平均梯度。

具体示例

假设目标等效 Batch Size 为 32，但硬件限制单次最大 Batch Size 为 2。

- **设置参数**：‘micro_batch_size = 2’; ‘gradient_accumulation_steps = 16’ (因为 $2 * 16 = 32$)
- **执行流程**：
 1. 输入 2 个样本，计算 Loss 并除以 16。执行 ‘backward()’，计算出的梯度存入 ‘.grad’。不更新权重。
 2. 输入接下来 2 个样本，计算 Loss 并除以 16。执行 ‘backward()’，新梯度与 ‘.grad’ 中原有的梯度相加。**不更新权重。**
 3. ... (重复此过程)
 4. 输入最后 2 个样本，计算 Loss 并除以 16。执行 ‘backward()’。此时 ‘.grad’ 中完整包含了全部 32 个样本的平均梯度。
 5. 执行 ‘optimizer.step()’ 修改模型权重，随后执行 ‘optimizer.zero_grad()’ 清空累计梯度。

最终结果：内存消耗始终保持在 Batch Size = 2 的水平，但模型优化的轨迹与 Batch Size = 32 完全一致。

13.4. DPO 训练实验

终于来到了最后的环节。

训练 DPO 需要两个 GPU(如果一个 GPU 显存够大放一个也行)，一个用来放冻结的 SFT 起始模型副本，也就是公式里的 π_{ref} ，它是不被更新的模型，只是用来限制要被更新的 π_{theta} 不要偏离基座模型太多。

我是租的 PRO 6000 一张 96G 的两张卡，设置最大长度 $\text{max-seq-len}=2048$ ， $\text{micro-batch-size}=8$ ， $\text{gradient-accumulation-steps}=8$ 。差不多第一张用来训练的卡 96G 能在峰值吃满，第二张用来加载基座模型的卡峰值大概是 25G 左右。训练总时间是 3 个小时左右，训练了两次一共花了 70 多块。

一开始我想着就按照 hh-rlhf 里的那样按照现成的 train 和 test 那样划分训练和验证集，但是我看讲义让把训练集再分出来一部分作为验证集，那我就按照他的做了。学习率一开始我是按讲义里那样设置的 $1e-6$ ，后面又跑了一版 $2e-6$ ，效果是更好一些的。

具体到实验结果上，

训练集正常走低，验证集虽然损失下降比较低，但也是往下走的，不过具体到验证集的准确率上没啥太大起色。

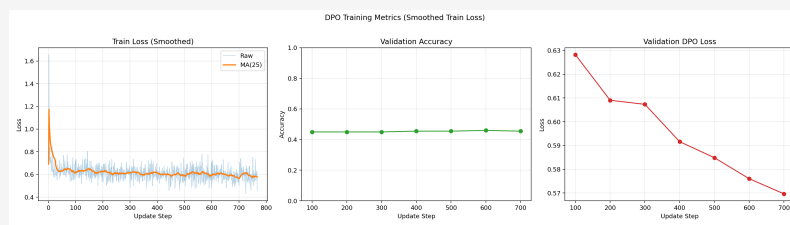


图 13.3: DPO 训练结果

到最后的任务评估上也能看出，gsm8k 和 mmlu 相较于 SFT 之后的模型都只是小幅进步，(吞吐量变快了是因为我换了别的卡评估的，是硬件能力变强了)。从格式上来看，gsm8k 数据解析成功率更好了，mmlu 解析成功率依旧糟糕，但我看了一下客观结果，感觉还是有进步的。

后面两个感觉也没啥大提升。。

13.4.1. 最终评估结果

任务	指标	基座模型	指令微调后模型	DPO 后模型
gsm8k	样本数量	1319	1319	1319
	准确率	10.462%	18.423%	18.95%
	解析失败数量	13	9	3
	解析失败率	0.986%	0.682%	0.23%
	样本吞吐量	0.971	0.981	9.218
mmlu	样本数量	14042	14042	14042
	准确率	58.375%	54.878%	55.00%
	解析失败	29	239	294
	解析失败率	0.207%	1.702%	2.09%
	样本吞吐量	0.749	5.263	31.281
alpaca_eval	样本数量	805	805	805
	样本吞吐量	0.405	0.433	4.31
simple_safety_tests	样本数量	100	100	100
	样本吞吐量	0.740	0.472	5.63